Breaking the IEEE Encryption Standard: XCB-AES in Two Queries

Amit Singh Bhati, COSIC, KU Leuven; 3MI Labs, Belgium
Elena Andreeva, TU Wien, Austria
Crypto 2025

Venkatesh Kumar
251110617
Dept. of CSE, IIT Kanpur

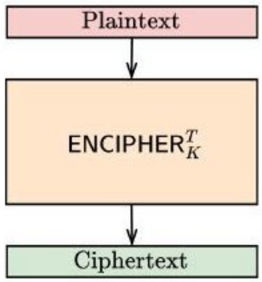
CS641: Modern Cryptology
Instructor: Prof. Somitra Sanadhya

May 2, 2026

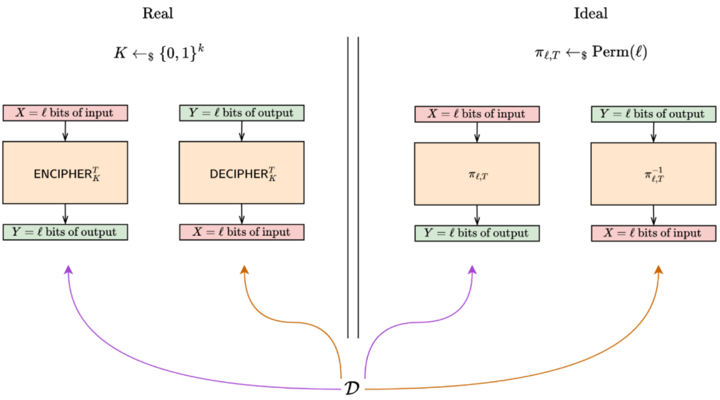
Outline

Tweakable Enciphering Modes (TEMs)

- Generalization of (tweakable) block ciphers
- Supports variable input size and tweak
- Length-preserving encryption
- Applications:
  - Disk sector and full disk encryption
  - Key wrapping and Swap-file encryption
  - Robust AEAD



Security of TEMs

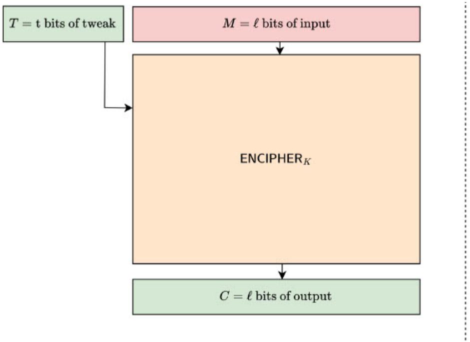


- (VIL-STPRP)Variable-Input-Length Strong Tweakable PRP
- Equivalent to strong IND-CCA style guarantees
- Ensures indistinguishability from random permutation

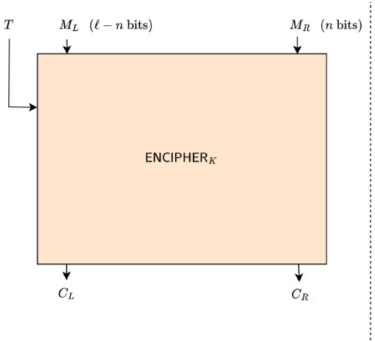
XCB-AES Standard

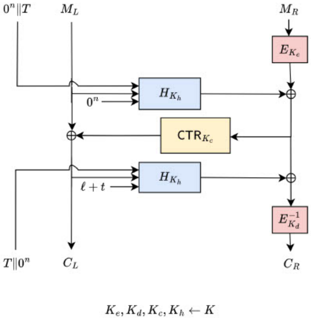
- IEEE 1619.2 standard (2010, updated 2021)
- Designed for storage encryption
- Efficient Hash-CTR-Hash construction
- XCB-AES Timeline
  - 2010: Standardized
  - 2013: Padding attack discovered
  - 2021: Updated to XCBv2 (padding-free)
  - Claimed security up to birthday bound
    - Proven VIL-STPRP up to birthday bound for block-aligned messages
    - Claimed security up to birthday bound $2^{(52-\log l)}$ Queries
  - 2024: They break XCB-AES's VIL-STPRP, STPRP and SPRP security in 2 queries. It applies to all other XCB-style modes as well

XCB-AES: IEEE 1619.2 TEM Standard



XCB-AES: IEEE 1619.2 TEM Standard

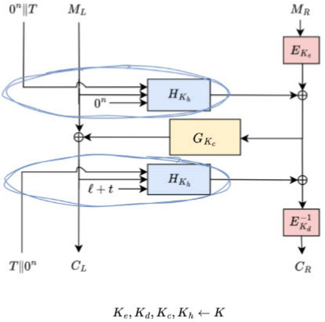




- CTR = Counter mode
- E = AES blockcipher
- H = a polynomial/rolling hash e.g., Polyval, GHASH

$$Poly_K(A_1 \| A_2 \| \dots) = A_1 K \oplus A_2 K^2 \oplus \dots$$

Their Shared Difference Attack



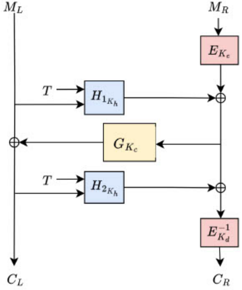
- $H_{K_h}(0^n \| T, M_L, 0^n)$

  $=\text{Poly}_{K_h}(0^n \| pad_n(T) \| Pad_n(M_L) \| 0^n)$

- $H_{K_h}(T \| 0^n, C_L, l + t)$

  $=\text{Poly}_{K_h}(pad_n(T) \| 0^n \| pad_n(C_L) \| bin_n(l + t))$

Their Shared Difference Attack



$K_e, K_d, K_c, K_h \leftarrow K$

- $H_{K_h}(0^n \| T, M_L, 0^n)$

$=\mathsf{Poly}_{K_h}(0^n \| pad_n(T) \| Pad_n(M_L) \| 0^n)$

$=\mathsf{H}_1(K_h, T, M_L)$

- $H_{K_h}(T \| 0^n, C_L, l+t)$

$=\mathsf{Poly}_{K_h}(pad_n(T) \| 0^n \| pad_n(C_L) \| bin_n(l+t))$

$=\mathsf{H}_2(K_h, T, C_L)$

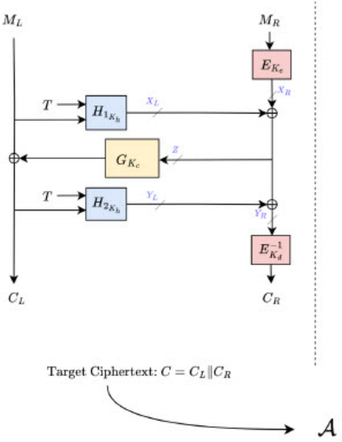
Polynomial Hash Separability

$$Poly_K(A_1 \| A_2 \| (A_3 \oplus \Delta) \| A_4) = A_1 K \oplus A_2 K^2 \oplus (A_3 \oplus \Delta) K^3 \oplus A_4 K^4$$
$$= A_1 K \oplus A_2 K^2 \oplus A_3 K^3 \oplus A_4 K^4 \oplus \Delta K^3$$
$$= Poly_K(A_1 \| A_2 \| A_3 \| A_4) \oplus Poly_K(0^n \| 0^n \| \Delta \| 0^n)$$

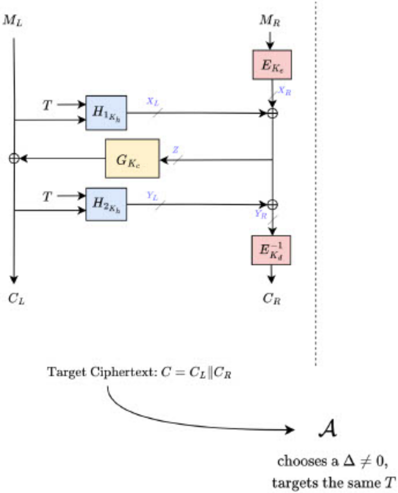
Polynomial Hash Separability

$$H_1(K_h, T, M_L \oplus \Delta) = Poly_{K_h}(0^n \| pad_n(T) \| pad_n(M_L \oplus \Delta) \| 0^n)$$
$$= \mathsf{Poly}_{K_h}(0^n \| pad_n(T) \| pad_n(M_L) \| 0^n) \oplus Poly_{K_h}(0^n \| 0^{|pad_n(T)|} \| 0^x \| \| \Delta \| 0^y \| 0^n)$$
$$= H_1(K_h, T, M_L) \oplus f_\Delta$$

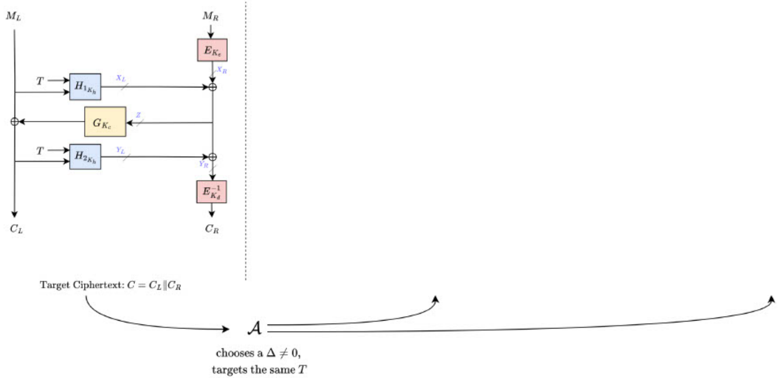
$$H_2(K_h, T, C_L \oplus \Delta) = Poly_{K_h}(pad_n(T) \| 0^n \| pad_n(C_L \oplus \Delta) \| bin_n(l+t))$$
$$= \mathsf{Poly}_{K_h}(pad_n(T) \| 0^n \| pad_n(C_L) \| bin_n(l+t)) \oplus Poly_{K_h}(0^{|pad_n(T)|} \| 0^n \| 0^x \| \| \Delta \| 0^y \| 0^n)$$
$$= H_2(K_h, T, C_L) \oplus f_\Delta$$


M_L
M_R
E_{K_e}
X_R
T
H_{1K_h}
X_L
G_{K_c}
Z
T
H_{2K_h}
Y_L
Y_R
$E^{-1}_{K_d}$
C_L
C_R
Target Ciphertext: $C = C_L \| C_R$
$\mathcal{A}$

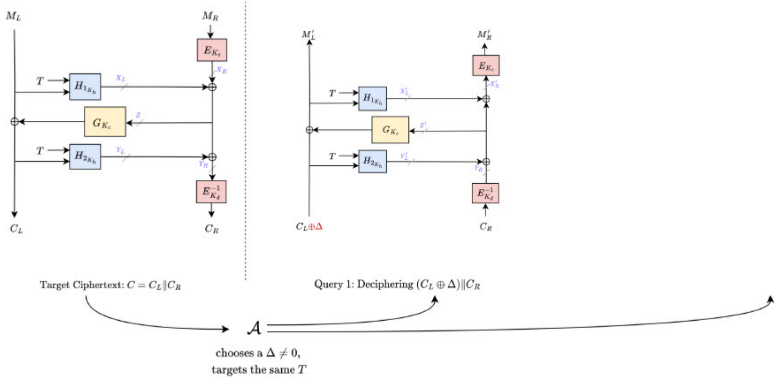
Their Shared Difference Attack



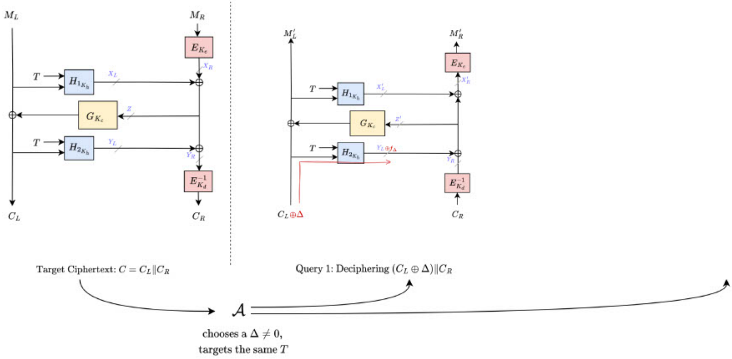
Their Shared Difference Attack



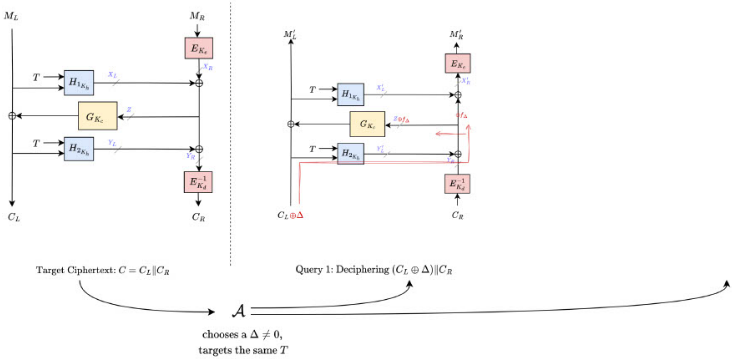
Their Shared Difference Attack



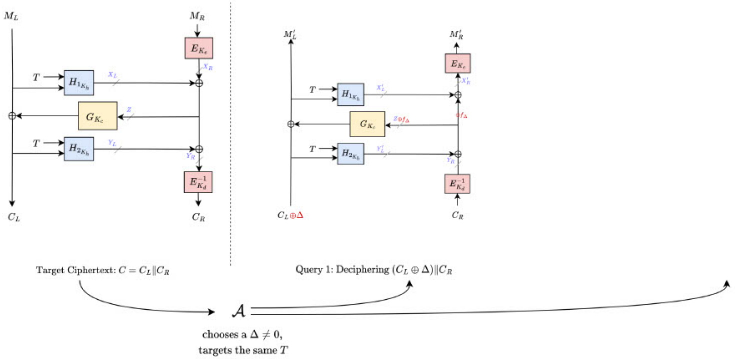
Their Shared Difference Attack



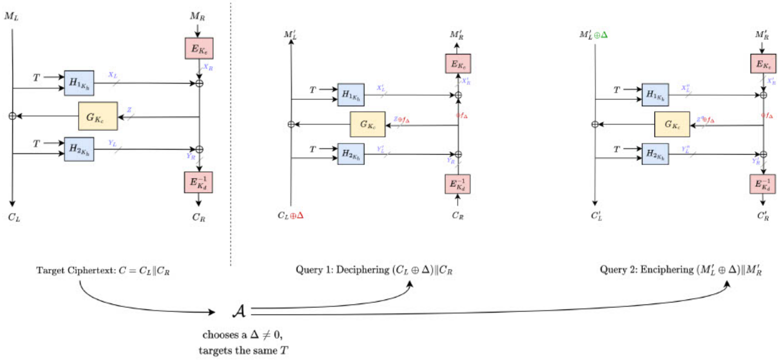
Their Shared Difference Attack



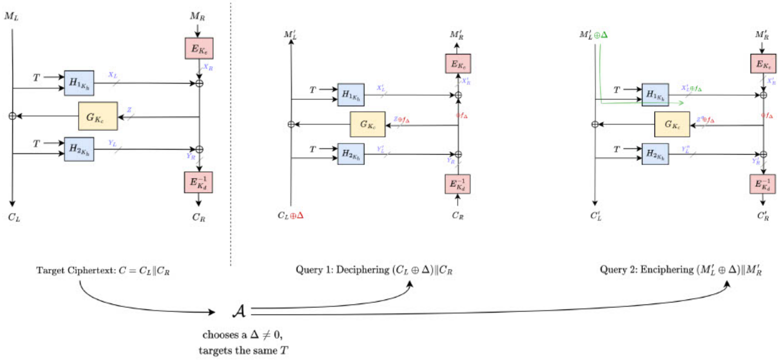
Their Shared Difference Attack



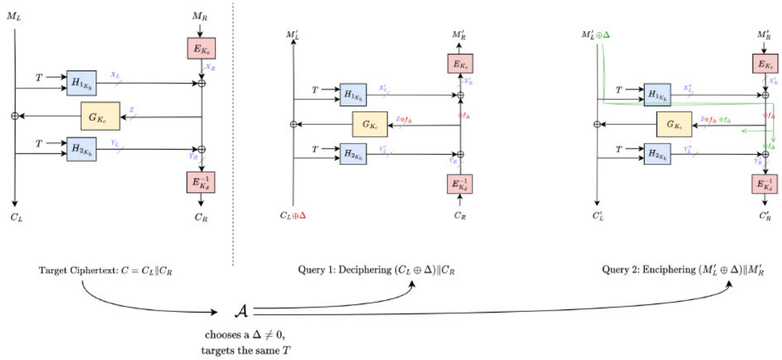
Their Shared Difference Attack



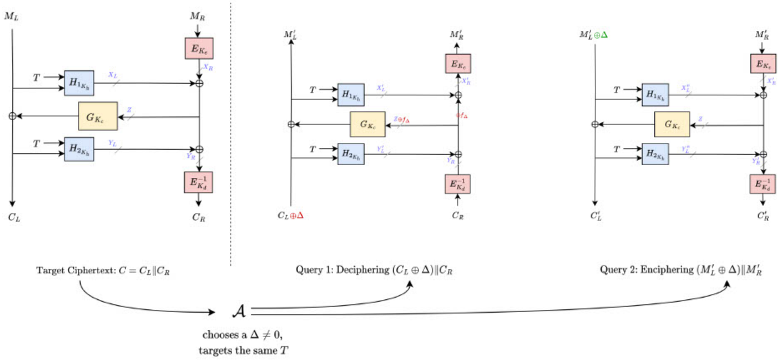
Their Shared Difference Attack



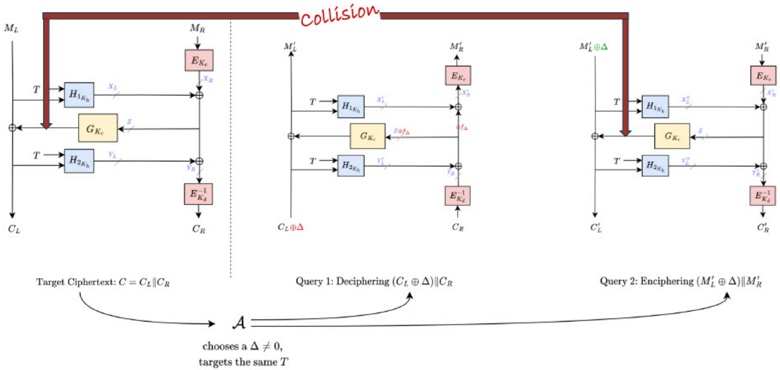
Their Shared Difference Attack



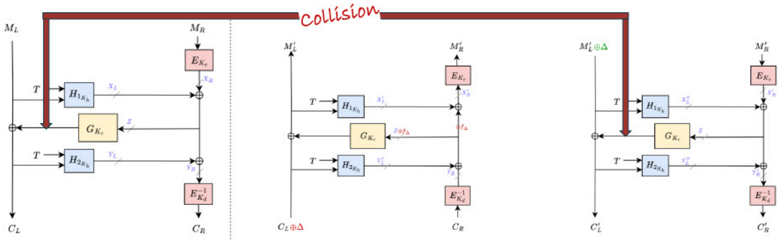
Their Shared Difference Attack



Their Shared Difference Attack

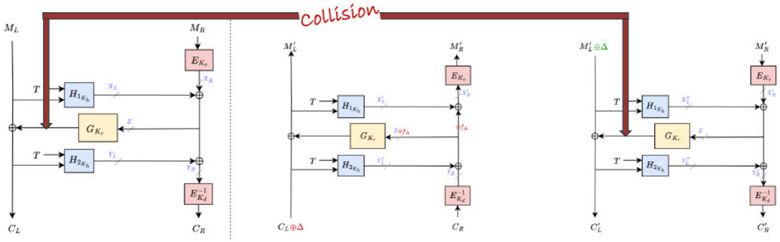


Their Shared Difference Attack



$$M_L \oplus C_L = (M_L' \oplus \Delta) \oplus C_L'$$

Their Shared Difference Attack



$$M_L \oplus C_L = (M'_L \oplus \Delta) \oplus C'_L$$
$$M_L = (M'_L \oplus \Delta) \oplus C'_L \oplus C_L$$

Message Recovered

Why the Attack Works

Root Cause: A Shared Difference Property

Shared Difference Property of Polynomial Sum

$$H_{sum}(K,T,M,C) = H_1(K,T,M) \oplus H_2(K,T,C)$$
$$H_{sum}(K,T,M \oplus \Delta, C \oplus \Delta) = H_1(K,T,M \oplus \Delta) \oplus H_2(K,T,C \oplus \Delta)$$
$$= H_1(K,T,M) \oplus f_\Delta \oplus H_2(K,T,C) \oplus f_\Delta$$
$$= H_1(K,T,M) \oplus H_2(K,T,C)$$

Due to separability of polynomial hash

$$H_{sum}(K,T,M \oplus \Delta, C \oplus \Delta) = H_{sum}(K,T,M,C)$$

Flaw in Existing Proofs

- Assumption:
  - Hash functions are XOR-universal
- Claimed:
  - Assumes H_1, H_2 are XOR-universal
  - ~~Implies H_{sum} is universal~~
  - CTR IV unpredictable and hard to collide
  - Independent and random CTR key streams up to birthday bound
- Reality:
  - Sum is **not XOR-universal**

Flaw in Existing Proofs

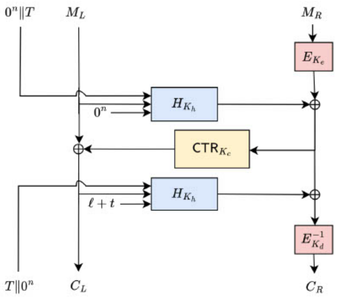
- For any two inputs $(T, M) \neq (T', M')$, output Y, and random secret key K,
  $Pr(H_1(K, T, M) \oplus H_1(K, T', M') = Y) \leq l/2^n$
  H_1 is XOR-universal
- For any two inputs $(T, C) \neq (T', C')$, output Y, and random secret key K,
  $Pr(H_2(K, T, M) \oplus H_2(K, T', M') = Y) \leq l/2^n$
  H_2 is XOR-universal
- For any two inputs $(T, M, C) \neq (T', M', C')$, output Y, and random secret key K,
  $Pr(H_{sum}(K, T, M, C) \oplus H_{sum}(K, T', M', C') = 0) = 1$
  $M' = M \oplus \Delta, C' = C \oplus \Delta$
  H_{sum} is not XOR-universal

Results Summary

Attack	Schemes	Message length	Attack type	# queries
LRW1 CCA attack by Khairallah [Kha23]	XCBv1, XCBv2, XCBv2fb	n bits	recovery of n bits	3
Shared difference attack [This work]	XCBv1	all $m > n$ bits	recovery of $m - n$ bits	4
	XCBv2, XCBv2fb, HCI, MXCB	all $m > n$ bits	recovery of $m - n$ bits	2
Flipped parts attack [This work]	HCI	all $m > n$ bits	distinguishing attack	3

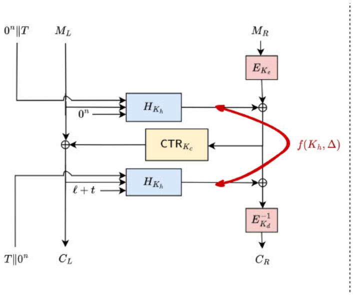
m can be arbitrarily large

Repairing and Enhancing XCB

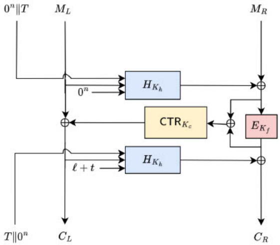


- XCB style

Repairing and Enhancing XCB



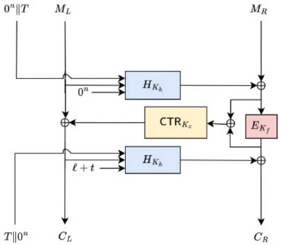
- XCB style
  - Insecure in current form.



Avoid the Sum

- XCB style
  - Insecure in current form.

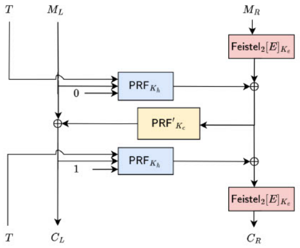
Repairing and Enhancing XCB



Avoid the Sum

- XCB style
  - Insecure in current form.
- HCTR2 style
  - AES-128, PolyVal
  - 64-bit STPRP security
  - 1.1 cpb on Gracemont

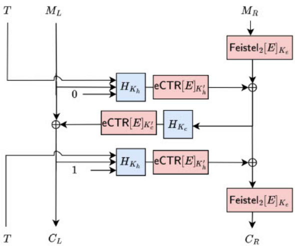
Repairing and Enhancing XCB



Use Inseparable Hashes

- XCB style
    - Insecure in current form.
- HCTR2 style
    - AES-128, PolyVal
    - 64-bit STPRP security
    - 1.1 cpb on Gracemont
- GEM style

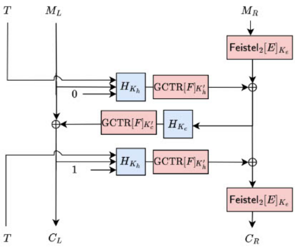
Repairing and Enhancing XCB



Use Inseparable Hashes

- XCB style
  - Insecure in current form.
- HCTR2 style
  - AES-128, PolyVal
  - 64-bit STPRP security
  - 1.1 cpb on Gracemont
- GEM style
  - AES-128, PolyVal
  - 128-bit STPRP security
  - 1.4 cpb on Gracemont

Repairing and Enhancing XCB



Use Inseparable Hashes

- XCB style
  - Insecure in current form.
- HCTR2 style
  - AES-128, PolyVal
  - 64-bit STPRP security
  - 1.1 cpb on Gracemont
- GEM style
  - Butterknife, PolyVal
  - 128-bit STPRP security
  - 1.1 cpb on Gracemont

Conclusion

- Their work introduced a shared difference attack against XCB
  - Breaking SPRP, STPRP, VIL-STPRP security of all XCB variants
  - Including XCB-AES; a 15 year old IEEE standard
- Pinpointed exact flaw in existing analyses
- Presented some countermeasures – HCTR2 and GEM

Impact

IEEE has officially removed XCB-AES from 1619.2 standard

Takeaway

Efforts toward TEM design and analysis through NIST's accordion initiative are essential

Thank You

Questions?